

MI IQSERVER API

Version 3.0

© 2021 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
3.0	Initial release	12/04/2020

TABLE OF CONTENTS

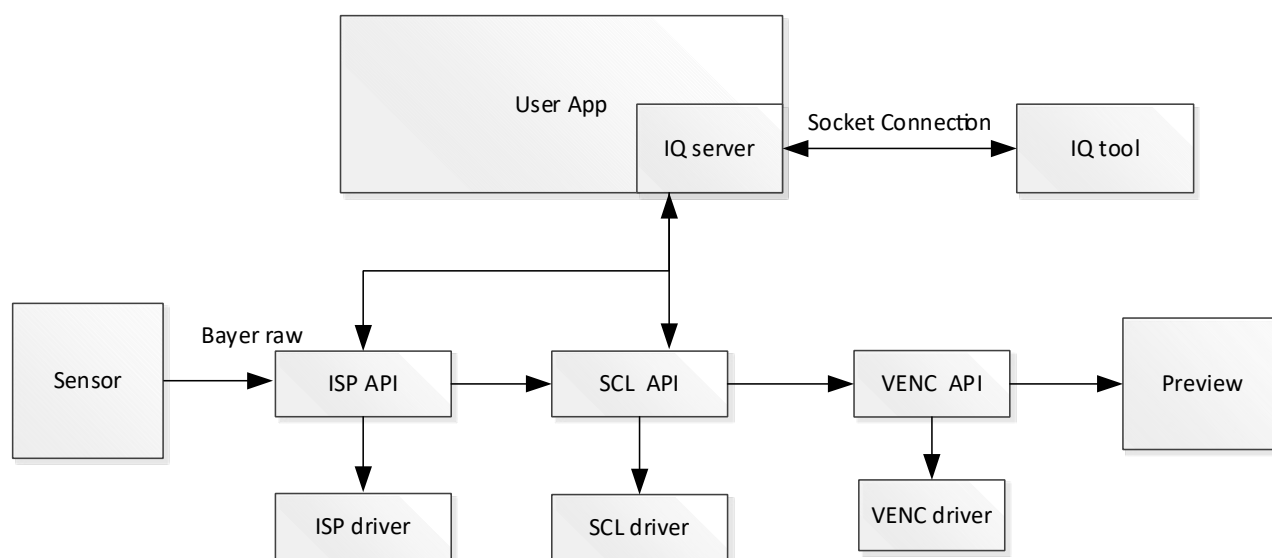
REVISION HISTORY.....	i
TABLE OF CONTENTS.....	ii
1. SUMMARY.....	1
1.1. Module Description.....	1
1.2. Flow Chart.....	1
1.3. Keyword.....	1
2. API LIST.....	2
2.1. MI_IQSERVER_Open.....	2
2.2. MI_IQSERVER_Close.....	3
2.3. MI_IQSERVER_SetDataPath.....	4
2.4. MI_IQSERVER_SetCustFunc.....	5
3. IQSERVER Error code.....	7

1. SUMMARY

1.1. Module Description

IQSERVER (Image Quality tuning Server) is used for data transport between tuning tool (IQ Tool) and EVB, including ISP parameter setting, image capture and file upload/download.

1.2. Flow Chart



1.1. Keyword

- **ISP**
Image Signal Processing, used for image noise reduction, color rendering, brightness adjustment, and other functions.
- **SCL**
Scale.
- **VENC**
Video Encoder, used for encoding YUV stream to H264/H265/Jpeg format bit-stream.

2. API LIST

This function module provides the following APIs:

API Name	Function
MI_IQSERVER_Open	Open IQ server
MI_IQSERVER_Close	Close IQ server
MI_IQSERVER_SetDataPath	Set IQ server data path
MI_IQSERVER_SetCustFunc	Set user specified execute function

2.1. MI_IQSERVER_Open

➤ Function

Call this interface to open IQ server.

➤ Syntax

```
MI_S32 MI_IQSERVER_Open();
```

➤ Form parameter

Parameter Name	Description	Input/Output
----------------	-------------	--------------

➤ Return Value

- MI_IQSERVER_OK: Successful
- Others: Failed, see [error code](#) for details.

➤ Dependency

- Head file: mi_iqserver.h
- Others: libmi_iqserver.a

➤ Note

Currently, only one IQ server can be created. After creation, you can use IQ Tool to connect and debug.

➤ Example

```
/*declaration*/  
MI_S32 s32Ret = 0;  
/* open IQSERVER*/  
s32Ret = MI_IQSERVER_Open();
```

```

if (MI_IQSERVER_OK != s32Ret)
{
    Printf("open iqserver error:%x\n", s32Ret);
    return s32Ret;
}

```

➤ Related APIs

[MI_IQSERVER_Close](#)

2.2. MI_IQSERVER_Close

➤ Function

Call this interface to close IQ server.

➤ Syntax

```
MI_S32 MI_IQSERVER_Close();
```

➤ Form parameter

Parameter Name	Description	Input/Output
----------------	-------------	--------------

➤ Return Value

- MI_IQSERVER_OK: Successful.
- Others: Failed, see [error code](#) for details.

➤ Dependency

- Head file: mi_iqserver.h
- Library file: libmi_iqserver.a

➤ Note

The number of calls for [MI_IQSERVER_Open](#) and [MI_IQSERVER_Close](#) must correspond.

➤ Example

```

/*declaration*/
MI_S32 s32Ret = 0;
/* close IQSERVER*/
s32Ret = MI_IQSERVER_Close();
if (MI_IQSERVER_OK != s32Ret)
{
    Printf("close iqserver error:%x\n", s32Ret);
    return s32Ret;
}

```

➤ Related APIs

[MI_IQSERVER_Open](#)

2.3. MI_IQSERVER_SetDataPath

➤ Function

Set IQ server data path.

➤ Syntax

```
MI_S32 MI_IQSERVER_SetDataPath(char* path);
```

➤ Form parameter

Parameter Name	Description	Input/Output
path	Path of IQ server data, ending with \0.	Input

➤ Return Value

- MI_IQSERVER_OK: Successful.
- Others: Failed, see [error code](#) for details.

➤ Dependency

- Head file: mi_iqserver.h
- Library file: libmi_iqserver.a

➤ Note

- You can call this API before [MI_IQSERVER_Open](#).
- Before calling this API, make sure that the path is accessible and that the data needed by the IQ server is available.
- Normally, the data needed by the IQ server include isp_api.xml (for use by IQ Tool).

➤ Example

```
/*declaration*/
MI_S32 s32Ret = 0;
char *data_path = "/customer/iqconfig/";
/* set IQSERVER data path*/
s32Ret = MI_IQSERVER_SetDataPath(data_path);
if (MI_IQSERVER_OK != s32Ret)
{
    Printf("set iqserver data path error:%x\n", s32Ret);
    return s32Ret;
}
```

➤ Related APIs

N/A.

2.4. MI_IQSERVER_SetCustFunc

➤ Function

Set user-specified execute function, for IQ server to call it.

➤ Syntax

```
MI_S32 MI_IQSERVER_SetCustFunc(MI_S32(*func)(MI_U16 data_type, MI_U32 length,
MI_U8 *data));
```

➤ Form parameter

Parameter Name	Description	Input/Output
func	Execute function	Input

➤ Return Value

- MI_IQSERVER_OK: Successful.
- Others: Failed, see [error code](#) for details.

➤ Dependency

- Head file: mi_iqserver.h
- Library file: libmi_iqserver.a

➤ Note

- Before using this API, define the function with the parameters below:
 1. datatype: type of data, user defined, to distinguish different data.
 2. length: length of data, IQ tool will auto set the value.
 3. data: data address.

You can do some processing in func with these parameters.
- By this API, you can add user-defined execute function in IQ server. IQ server will get the data from IQ tool, then call the user-defined function. After that, func will get the data, to implement custom features.
For example:
 1. Use IQ tool to write firmware, then IQ server calls func to flash firmware.
 2. Use IQ tool to set IR cut, then IQ server calls func to switch IR.
- Before calling this function, make sure func is executable.
- Data_type and data come from IQ tool, and length means data length.
- Data_type can be used to distinguish what function the data is used for. You can define different function for different data.

➤ Example

```
/*declaration*/
MI_S32 s32Ret = 0;
MI_S32 sample_func(MI_U16 data_type, MI_U32 length, MI_U8 *data)
{
    printf("call MI_IQSERVER_SetCustFunc() success\n");
    printf("data_type is %d, length:%u, data addr:%p\n", data_type, length, data);
}
```

```
        //do something
    return MI_IQSERVER_OK;
}
/* set IQSERVER custom function*/
s32Ret = MI_IQSERVER_SetCustFunc(sample_func);
if (MI_IQSERVER_OK != s32Ret)
{
    Printf("set iqserver cust func error:%x\n", s32Ret);
    return s32Ret;
}
```

- Related APIs
N/A.

3. IQSERVER ERROR CODE

The IQSERVER API error code is as follows:

Error Code	Macro Definition	Description
0x0	MI_IQSERVER_OK	Successful
0x1	MI_ERR_IQSERVER_FAIL	Failed
0x2	MI_ERR_IQSERVER_NULL_PTR	Null pointer
0x3	MI_ERR_IQSERVER_NO_BUFF	No buffer